

匹克球四类击球分类的稀疏混合专家模型设计与实验方案

执行摘要

本任务可以明确建模为**单次击球级别的表格型四分类问题**：输入是每次击球的数值特征，输出是四类击球标签。对这类数据，先做一个**强基线的稠密 MLP**非常必要，因为表格深度学习文献反复强调，简单而调优充分的 MLP/MLP-like 模型仍然是必须对照的基线，而且在不少数据集上并不弱；同时，MoE 的优势更偏向于输入空间存在明显异质子区域、希望通过条件计算形成“专家分工”的场景。对于学生作业、单机训练、小到中等规模数据，MoE 的主要价值往往不是纯速度，而是**可解释的分工现象**与“同样前向只激活少数子网络”的建模思路。¹

从可交付角度，建议把作业实现压缩为三组模型：**稠密 MLP、MoE-K=1、MoE-K=2**。数据侧采用“先切分、再拟合预处理器”的原则：若有比赛/回合/球员 ID，则优先做**group-aware**切分；若无，则做**分层 70/15/15**的 train/val/test。数值特征先检查缺失值，再用训练集拟合中位数填补与标准化；类别如果不平衡，优先用**加权交叉熵**，必要时再考虑仅对训练集做 SMOTE 或保守的数值增强。这样能同时满足性能、可复现性和避免泄漏。²

模型侧建议采用一个适合作业实现的“小型稀疏 MoE”版本：共享前端 stem 提取基础表征，随后进入门控网络与多个专家；专家数默认设为**4**，先做**K=1**与**K=2**；负载平衡项以**Switch Transformer 风格的辅助损失**作为主实现，若想与原始论文更贴近，则附加实现**Shazeer 的 importance/load CV² 损失**作为扩展版。对 4 页实验报告而言，最重要的是把结果收束到一张总表、一张混淆矩阵、一张专家分工热图和一段对负载平衡必要性的讨论。³

如果时间有限，**最低可交付版本**建议是：先把数据流水线、MLP、MoE-K=1 跑通；再在同一份代码上切换到 K=2；最后补上路由统计、混淆矩阵和专家热图。这样既能保证代码完整，也能保证报告里有清晰的实验对比逻辑。⁴

任务界定与文献基础

本题已知输入是“每次击球的数值特征”和“四类标签”，因此方法上更接近**表格分类**而不是视频动作识别。结合匹克球技术与飞行动力学研究，击球方向、站位、上网速度、球速、落点位置、击球发射点/高度/角度等变量都与打法和结果高度相关，因此你题目中给出的站位、挥拍速度、球速、落点 x/y 等特征，在语义上是合理且有判别力的。⁵

MoE 的经典思想最早来自早期 mixture-of-experts 体系，核心是用一个门控网络把输入动态分给不同专家；而稀疏 MoE 的关键进展在于，只激活少数专家，从而把“参数容量”与“每样本计算量”解耦。Shazeer 等人在 2017 年把这种思路系统化为稀疏门控专家层，GShard 把 top-2 路由与容量控制带到大规模训练中，Switch Transformer 又进一步把 top-1 路由做得更简单、更稳定。中文综述也强调了 MoE 的四个关键优势：扩展性、低计算开销、专业化能力与泛化性。⁶

对你的作业而言，最重要的不是追求大规模并行，而是借用这些文献里**最小必要的机制**：门控网络、Top-K 稀疏选择、负载平衡项、专家分工分析。与此同时，要保持一个现实判断：对于单机、小样本、结构化数据任务，稀疏 MoE 不一定天然优于稠密模型；Hugging Face 的中文 MoE 说明也明确提出，稀疏模型通常更适合多

机高吞吐场景，而资源较少、吞吐要求不高时，稠密模型可能更合适。因此，这个作业的实验重点应放在**是否出现可解释的专家分工**以及**K=1/K=2 是否优于强 MLP 基线**，而不是预设 MoE 一定胜出。 7

下面这张表把题目中的已知条件与未指定条件收拢出来，后续实现都以它为前提。

项目	状态	处理建议
每次击球的数值特征	已知	按表格数据处理
四类标签	已知	编码为 0,1,2,3
具体四类名称	未指定	报告中先用 C0-C3，或在代码里读 label map
样本总量	未指定	决定模型宽度、专家数、是否需要 stronger regularization
缺失值情况	未指定	进入预处理前先统计缺失率
类别是否不平衡	未指定	先画 label 分布，再决定 class weight / SMOTE
是否有比赛/回合/球员 ID	未指定	若有，优先 group-aware 划分避免泄漏
时间序列上下文	未指定	本报告按“单次击球独立分类”实现；若有上下文可作为扩展

数据准备与预处理

预处理的第一原则是**先划分，再拟合预处理器**。scikit-learn 明确把训练/测试混用造成的信息泄漏列为常见错误，并强调切分应该发生在预处理之前；`train_test_split` 则支持对分类任务使用 `stratify=y` 保持类别比例。如果你的数据中存在比赛 ID、回合 ID 或球员 ID，这一原则还应进一步升级为 **group-aware split**：同一比赛或同一球员的样本不要同时进入训练集和测试集，否则会产生过于乐观的泛化估计。对这类分组且多分类的情况，可以优先考虑 `StratifiedGroupKFold` 或用 `GroupShuffleSplit` 做 holdout。 8

对于数值型击球特征，建议的默认流程是：先检查异常值与单位，再做缺失值填补，再做标准化。缺失值如果存在且比例不高，优先用**训练集拟合的中位数填补**；标准化则用 `StandardScaler`，其定义是 $z = (x - u)/s$ ，并且其中的均值与标准差都应来自训练集。对于 MLP 与 MoE 这种基于梯度和矩阵乘法的模型，标准化通常比“保留原始单位”更稳。 9

就你题目给的特征类型，除原始特征外，建议再补几类**轻量特征工程**，但不要做太重：第一类是几何特征，如“距球网距离”“落点到中线/边线距离”“落点所在区域 one-hot 或 ordinal bin”；第二类是运动关系特征，如“球速/挥拍速度比值”“落点纵深”“横向角度变化”；第三类是 if available（未指定）的上下文差分特征，例如与上一拍的站位变化、球速变化、落点变化。这些特征都和匹克球中的方向变化、上网节奏、深浅落点、快慢节奏高度一致。 5

数据增强方面，不建议一上来做复杂跨类插值增强。更稳妥的顺序是：先看类别分布，若确实不平衡，优先使用**加权交叉熵**；若长尾很明显，再尝试**仅在训练集使用 SMOTE**。PyTorch 的 `CrossEntropyLoss` 允许为每个类别传入权重，官方文档明确说明它对不平衡训练集特别有用；而 balanced accuracy 则是把每类召回率做平均，适合在你怀疑“总体准确率被大类冲高”时一起观察。SMOTE 是经典合成过采样方案，但在物理约束明显的运动特征上要谨慎使用，因为插值样本可能落到不合理区域。 10

如果你仍然希望写“数据增强”这一项，建议在报告里使用非常克制的表述：**仅对连续特征做小幅高斯抖动**，且必须限定在物理合理范围内，例如落点 x/y 不能越界，球速不能为负，挥拍速度与球速比值不能到异常极端。对于课程作业，这样写通常比“激进增强却解释不清”更稳。 11

下面给出一个适合作业直接照搬的预处理清单。

步骤	默认建议	备注
标签编码	<code>{'class_name': 0..3}</code> 或直接 0..3	四类名称未指定时用 C0-C3
划分	70/15/15	若有 group 字段，优先按 group 切分
缺失值	<code>SimpleImputer(strategy='median')</code>	仅在训练集 <code>fit</code>
标准化	<code>StandardScaler()</code>	仅在训练集 <code>fit</code>
类别不平衡	先 <code>class_weight</code> ，后再考虑 SMOTE	不建议一开始就同时重采样+高权重
可选特征工程	距网距离、落点区域、速度比值、角度特征	控制数量，避免报告过长
保存工件	<code>label_map.json</code> 、 <code>imputer.pkl</code> 、 <code>scaler.pkl</code>	便于复现实验

基线稠密 MLP 与 MoE 模型设计

对表格型任务，**强 MLP 基线不是走形式，而是必要对照**。RTDL (Revisiting Deep Learning Models for Tabular Data) 和其官方实现都强调：经过合理调优的简单 MLP 依然是好的基线，复杂方法并不总能稳定胜出。因此，建议先把一个结构规整、参数量不过大的稠密 MLP 跑扎实，再让 MoE 在同样的数据流水线和训练策略下比较。¹²

建议的稠密 MLP 结构如下。设输入维度为 d ，输出为 4 类。若样本量未指定、先求稳，默认使用三层隐藏层：`d -> 128 -> 64 -> 32 -> 4`。前两层可使用 `Linear + BatchNorm1d + ReLU + Dropout(0.2)`，最后一层隐藏层可用 `Linear + ReLU + Dropout(0.1)`，再接分类头。`BatchNorm1d` 官方文档说明它适用于 2D/3D 输入，并按 mini-batch 的特征维度统计均值和方差；在完整批次的稠密 MLP 里这是合理的。¹³

MoE 结构建议采用**共享 stem + 稀疏专家层 + 分类头**的形式，而不是直接让每个专家从原始输入全量起步。原因是：对你这种作业规模的数据，共享 stem 可以减少重复参数和过拟合风险，也让门控网络与专家都在一个更稳定的中间表示上工作。一个适合课程实现的默认版本是：

模块	默认结构
Shared Stem	<code>Linear(d, 64) -> BatchNorm1d(64) -> ReLU -> Dropout(0.1)</code>
GateNet	<code>Linear(64, 32) -> ReLU -> Dropout(0.1) -> Linear(32, E)</code>
Experts	<code>E=4</code> 个专家；每个专家： <code>Linear(64, 64) -> ReLU -> Dropout(0.1) -> Linear(64, 32) -> ReLU</code>
Combiner	Top-K 选中的专家输出做加权和
Classifier Head	<code>Linear(32, 4)</code>
K	进行 <code>K=1</code> 与 <code>K=2</code> 两版实验

这里建议**共享 stem** 可以用 `BatchNorm1d`，但**专家内部尽量不要再用 `BatchNorm1d`**。原因并不是 `BatchNorm` 本身不好，而是它依赖 mini-batch 统计；在 `K=1` 或 `K=2` 的稀疏路由下，每个专家每个 batch 实际

接收到的样本数可能很小且分布不均，批统计会更噪。对课程实现来说，专家内部用 `ReLU + Dropout` 更稳；如果你想再稳一些，可以换成 `LayerNorm`，但不是必需。`BatchNorm1d` 的文档明确说明其统计量按 mini-batch 计算，这正是做出这个工程判断的依据。 13

Shazeer 的稀疏门控写法是：先计算门控 logits，再保留 Top-K，其余位置置为 $-\infty$ ，最后只对保留的项做 softmax。原文还引入了 **Noisy Top-K Gating**，即在 softmax 之前向 logits 加可学习尺度的高斯噪声，以鼓励探索并改善负载平衡。GShard 在大规模 Transformer 里采用 top-2 路由；Switch 则把它进一步简化到 top-1，以降低通信和计算复杂度。对你的作业，一份足够扎实的设计是：默认实现“标准 softmax + Top-K”，把“噪声门控”作为进阶选项。 14

设 stem 输出为 $h \in \mathbb{R}^{64}$ ，专家数为 E ，门控网络输出 logits $z \in \mathbb{R}^E$ ，则推荐的作业版前向公式是：

$$p = \text{softmax}(z), \quad S = \text{TopK}(p, K)$$

$$\tilde{p}_i = \frac{p_i \cdot \mathbf{1}[i \in S]}{\sum_{j \in S} p_j + \epsilon}$$

$$u_i = E_i(h), \quad h_{\text{moe}} = \sum_{i=1}^E \tilde{p}_i u_i$$

$$\hat{y} = W h_{\text{moe}} + b$$

这样写的好处是，**K=1 与 K=2 共用同一套代码**，而且由于对选中的权重重新归一化，输出尺度更稳定。若你想更贴近 Switch 的写法，也可以在 K=1 版本里直接使用原始 top-1 gate 作为缩放项。MoE 输出由专家输出的加权和构成，这一点与 Shazeer 的 MoE 定义一致。 15

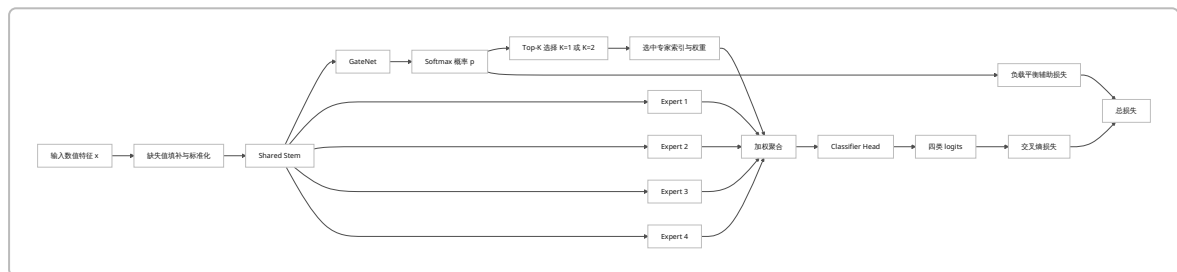
如果实现 Noisy Top-K，则推荐采用以下更贴近原始论文的写法：

$$H(x)_i = (xW_g)_i + \epsilon_i \cdot \text{softplus}((xW_{\text{noise}})_i), \quad \epsilon_i \sim \mathcal{N}(0, 1)$$

$$G(x) = \text{softmax}(\text{KeepTopK}(H(x), K))$$

对于单机课程作业，这个噪声头完全可以只在 `self.training` 时启用。若时间不够，先不实现噪声头也不影响主实验完成度。 16

下面的 mermaid 图可以直接放进实验报告，作为 MoE 数据流说明。



负载均衡项建议你在报告里写成“论文版”与“作业实现版”两层。Shazeer 原始方案里，先定义一个 batch 上专家的重要性：

$$\text{Importance}_i = \sum_{x \in X} G(x)_i$$

$$L_{\text{importance}} = w_{\text{importance}} \cdot CV(\text{Importance})^2$$

其中 $CV(v) = \frac{\text{std}(v)}{\text{mean}(v) + \epsilon}$ 。Shazeer 还进一步定义了平滑化的负载估计 Load，并给出：

$$L_{\text{load}} = w_{\text{load}} \cdot CV(\text{Load})^2$$

其出发点是：门控网络容易塌缩到“总是偏爱少数专家”的状态，而且这种不平衡会自我强化。 17

但在你的作业里，更建议**主实现**使用 Switch 风格的辅助损失，因为它更容易写、也更稳定。Switch 的 top-1 写法是：对 batch 中每个样本，统计每个专家被分配到的比例 f_i ，再统计路由概率均值 P_i ，最后定义

$$L_{\text{lb}} = \alpha \cdot E \cdot \sum_{i=1}^E f_i P_i$$

其中 E 是专家数。对 $K=2$ ，你可以做一个**作业级扩展**：把 f_i 改成“在全部 $B \times K$ 个已选专家位置里的归一化计数”，即

$$f_i = \frac{1}{BK} \sum_{b=1}^B \sum_{j=1}^K \mathbf{1}[\text{idx}_{b,j} = i]$$

而 P_i 仍然使用原始 softmax 概率在 batch 上的平均值。这个扩展不是 Switch 论文的原式，但很适合作业实现：简单、可微部分明确、 $K=1/K=2$ 统一。 18

$K=1$ 与 $K=2$ 的预期差别，建议在报告中明确写清楚：

| 路由方式 | 优点 | 风险 | 适合观察点 | |---|---|---| | $K=1$ | 计算最省、实现最简、统计最直观 | 更容易专家塌缩、边界样本表达偏硬 | 是否形成“一类一专家”或“区域专家” | | $K=2$ | 对模糊/混合样本更柔和，分工更细 | 计算更高，门控更分散，分析更复杂 | 第二专家是否主要服务于边界样本 |

这张表的理论依据分别来自 GShard 的 top-2 路由与 Switch 的 top-1 简化逻辑。 19

训练、评估与可视化方案

训练目标建议统一写成：

$$L_{\text{total}} = L_{\text{ce}} + \lambda_{\text{lb}} L_{\text{lb}} + \lambda_{\text{imp}} L_{\text{importance}}$$

其中主实验可令 $\lambda_{\text{imp}} = 0$ ，只保留 CrossEntropyLoss + Switch-style load balancing；扩展实验再打开 importance/load CV²。CrossEntropyLoss 官方文档明确说明其输入应是未归一化 logits，目标通常是类别索引，并且支持为每个类别设置权重；优化器推荐 AdamW，其权重衰减与一阶/二阶动量解耦。若验证集指标停滞，再用 ReduceLROnPlateau 自动降低学习率。 20

下面给出一个适合课程作业直接使用的默认超参数起点。它们不是“必须照抄”的固定答案，而是兼顾稳定性与实现成本的**起始配置**。其中学习率范围参考了 RTDL 官方复现仓库中 MLP 调优结果的统计，中位数约为 2.16×10^{-4} ，所以把初始学习率放在 $3e-4$ 左右是合理的。 21

项目	稠密 MLP	MoE
优化器	AdamW	AdamW

项目	稠密 MLP	MoE
初始学习率	3e-4	3e-4
<code>weight_decay</code>	1e-4	1e-4
Batch size	128	128
训练轮数上限	80	80
学习率调度	ReduceLROnPlateau	ReduceLROnPlateau
调度参数	<code>factor=0.5, patience=5</code>	<code>factor=0.5, patience=5</code>
早停	<code>patience=15, 监控 val_loss</code>	<code>patience=15, 监控 val_loss</code>
Dropout	0.1-0.2	stem 0.1, expert 0.1
<code>lambda_lb</code>	0	0.01 起试, 备选 0.001/0.05
随机种子	42	42

早停的概念可以直接按 PyTorch-Ignite 的定义来写：当监控指标在若干个 event 内没有改进时停止训练，`patience` 表示允许忍受的连续无改进次数。为了减少“偶然一次跑得很好”的争议，建议至少保留固定种子 42 的主结果；如果时间允许，再补 3 个种子（例如 42、52、62）的均值与标准差。需要注意，PyTorch 官方同时提醒：跨平台、跨版本、CPU/GPU 之间，完全可复现并无保证，所以报告里最好记录包版本、设备信息和随机种子。²²

评估指标方面，题目要求的核心指标是**准确率、混淆矩阵、每专家分配统计**；建议再加上 `classification_report` 的每类 precision / recall / F1，以及在类别不平衡时加一项 **balanced accuracy**。`accuracy_score` 是总体正确率；`confusion_matrix` 中第 i 行第 j 列表示真实类为 i 、预测为 j 的计数；`classification_report` 能直接给出每类与宏/加权平均指标；balanced accuracy 则是各类召回率的平均。²³

“每专家分配统计”建议至少记录四类信息。第一，**硬分配负载**：每个专家被选中的次数与占比。第二，**软重要性**：每个专家的平均路由概率 P_i 。第三，**专家-类别对应关系**：某类别样本有多大比例被路由到某专家。第四，**负载离散度**：例如 `CV(load)`、`max(load)/mean(load)`。Shazeer 在原文中就用 importance、load 及其 coefficient of variation 来衡量专家利用是否平衡。¹⁷

可视化方面，建议报告里至少放四类图。第一类是**训练曲线**：`train_loss`、`val_loss`、`val_acc`，若篇幅足够，再加 `lb_loss`。第二类是**混淆矩阵**，建议用按真实标签归一化的版本，这样每行和为 1，更容易看出哪一类最容易被混淆。第三类是**专家分工热图**，推荐两种：`class -> expert` 路由热图和 `expert -> class` 纯度热图。第四类是**门控概率分布图**，例如最大门控概率 `max(p)` 的直方图，或者门控熵 $H(p)$ 的分布。Matplotlib 官方提供了标注热图的标准写法，scikit-learn 也提供了 `ConfusionMatrixDisplay`。²⁴

热图最实用的定义方式有两个。若想回答“某类样本主要被送给谁”，就画

$$H_{c,e}^{(\text{class} \rightarrow \text{expert})} = \frac{\#\{b : y_b = c, e \in \text{TopK}_b\}}{\#\{b : y_b = c\}}$$

若想回答“某专家主要在处理哪类”，就画

$$H_{e,c}^{(\text{expert} \rightarrow \text{class})} = \frac{\#\{b : y_b = c, e \in \text{TopK}_b\}}{\#\{b : e \in \text{TopK}_b\}}$$

前者强调“类的去向”，后者强调“专家纯度”。如果两张图都对角线明显，说明路由在向“类对专家”的分工收敛；如果并不对角但按速度/落点区域出现块状模式，也仍然可能是有意义的“子模式专家”。这一点和 MoE 文献里“专家会自然形成专门化”的观察一致。²⁵

代码结构与关键伪代码

为了同时满足 `.py` 与 `.ipynb` 提交要求，一个适合作业的目录组织如下。

文件	作用
<code>notebook/eda_train.ipynb</code>	数据检查、一次性实验、图表导出
<code>src/data.py</code>	读表、划分、缺失值填补、标准化、Dataset/DataLoader
<code>src/models/mlp.py</code>	稠密 MLP
<code>src/models/moe.py</code>	Expert、GateNet、SparseMoE、负载均衡项
<code>src/train.py</code>	训练循环、验证、早停、checkpoint
<code>src/eval.py</code>	accuracy、classification_report、confusion_matrix、专家统计
<code>src/viz.py</code>	训练曲线、混淆矩阵、专家热图、门控分布图
<code>src/utlils.py</code>	<code>set_seed</code> 、日志、保存配置
<code>run_mlp.py</code>	运行 MLP 实验
<code>run_moe.py</code>	运行 MoE 实验
<code>configs/*.yaml</code>	超参数配置

下面的伪代码都按 **PyTorch + pandas + scikit-learn** 的最小可交付风格书写，重点是结构清晰、容易抄成真代码。

```
# src/data.py
def load_and_split(csv_path, label_col, feature_cols, group_col=None, seed=42):
    df = pd.read_csv(csv_path)

    # 仅示例：实际可加入异常值检查、单位检查
    X = df[feature_cols].copy()
    y = df[label_col].astype(int).to_numpy()

    if group_col is not None:
        groups = df[group_col].to_numpy()

    # 先 group-wise 切 test
    gss_1 = GroupShuffleSplit(n_splits=1, test_size=0.15, random_state=seed)
    train_val_idx, test_idx = next(gss_1.split(X, y, groups=groups))

    X_train_val, y_train_val = X.iloc[train_val_idx], y[train_val_idx]
    X_test, y_test = X.iloc[test_idx], y[test_idx]
    groups_train_val = groups[train_val_idx]
```

```

# 再 group-wise 切 val, 比例约等于总数据 15%
val_ratio_in_train_val = 0.15 / 0.85
gss_2 = GroupShuffleSplit(n_splits=1, test_size=val_ratio_in_train_val,
random_state=seed)
train_idx_local, val_idx_local = next(
    gss_2.split(X_train_val, y_train_val, groups=groups_train_val)
)

X_train = X_train_val.iloc[train_idx_local]
y_train = y_train_val[train_idx_local]
X_val = X_train_val.iloc[val_idx_local]
y_val = y_train_val[val_idx_local]

else:
    X_train, X_temp, y_train, y_temp = train_test_split(
        X, y, test_size=0.30, stratify=y, random_state=seed
    )
    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=seed
    )

imputer = SimpleImputer(strategy="median")
scaler = StandardScaler()

X_train = imputer.fit_transform(X_train)
X_train = scaler.fit_transform(X_train)

X_val = imputer.transform(X_val)
X_val = scaler.transform(X_val)

X_test = imputer.transform(X_test)
X_test = scaler.transform(X_test)

return {
    "X_train": X_train, "y_train": y_train,
    "X_val": X_val, "y_val": y_val,
    "X_test": X_test, "y_test": y_test,
    "imputer": imputer, "scaler": scaler
}

```

```

# src/models/mlp.py
class DenseMLP(nn.Module):
    def __init__(self, d_in, n_classes=4):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_in, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(0.2),

```



```

        nn.Linear(128, 64),
        nn.BatchNorm1d(64),
        nn.ReLU(),
        nn.Dropout(0.2),

        nn.Linear(64, 32),
        nn.ReLU(),
        nn.Dropout(0.1),

        nn.Linear(32, n_classes),
    )

    def forward(self, x):
        return self.net(x)

```

```

# src/models/moe.py
class Expert(nn.Module):
    def __init__(self, d_in=64, d_hidden=64, d_out=32):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_in, d_hidden),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(d_hidden, d_out),
            nn.ReLU(),
        )

    def forward(self, x):
        return self.net(x)

```

```

# src/models/moe.py
def topk_router(router_logits, k, noise_logits=None, training=False, eps=1e-9):
    # router_logits: [B, E]
    if training and noise_logits is not None:
        noise_std = F.softplus(noise_logits)
        router_logits = router_logits + torch.randn_like(router_logits) * noise_std

    probs = F.softmax(router_logits, dim=-1) # [B, E]
    topk_vals, topk_idx = torch.topk(probs, k=k, dim=-1) # [B, K], [B, K]

    # 用于前向聚合的归一化 top-k 权重
    topk_w = topk_vals / (topk_vals.sum(dim=-1, keepdim=True) + eps)

    # 稀疏概率矩阵，方便后续按专家收集权重
    sparse_probs = torch.zeros_like(probs)
    sparse_probs.scatter_(1, topk_idx, topk_w)

    return probs, sparse_probs, topk_idx, topk_w

```

```

# src/models/moe.py
def switch_style_load_balance_loss(router_probs, topk_idx, num_experts, k):
    """
    router_probs: [B, E], 未截断 softmax 概率
    topk_idx: [B, K], 硬路由结果
    """

    B, E = router_probs.shape

    # f_i: 每个专家被选中的归一化频率 (对 K=2 做 BK 归一化扩展)
    counts = torch.zeros(E, device=router_probs.device)
    flat_idx = topk_idx.reshape(-1)
    counts.scatter_add_(0, flat_idx, torch.ones_like(flat_idx, dtype=torch.float))
    f = counts / (B * k)

    # P_i: softmax 概率均值
    P = router_probs.mean(dim=0)

    # Switch top-1 原式是  $\alpha * E * \sum(f_i * P_i)$ 
    # 这里对 K>1 采用作业级自然扩展
    loss = E * torch.sum(f * P)
    return loss, {"f": f.detach(), "P": P.detach()}

```

```

# src/models/moe.py
class SparseMoE(nn.Module):
    def __init__(self, d_in, n_classes=4, num_experts=4, k=1, noisy_gating=False):
        super().__init__()
        self.num_experts = num_experts
        self.k = k
        self.noisy_gating = noisy_gating

        self.stem = nn.Sequential(
            nn.Linear(d_in, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(0.1),
        )

        self.gate = nn.Sequential(
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(32, num_experts),
        )
        self.noise_gate = nn.Linear(64, num_experts) if noisy_gating else None

        self.experts = nn.ModuleList([Expert(64, 64, 32) for _ in range(num_experts)])
        self.head = nn.Linear(32, n_classes)

    def forward(self, x):

```

```

h = self.stem(x) # [B, 64]
router_logits = self.gate(h) # [B, E]
noise_logits = self.noise_gate(h) if self.noisy_gating else None

router_probs, sparse_probs, topk_idx, topk_w = topk_router(
    router_logits, k=self.k, noise_logits=noise_logits, training=self.training
)

B = x.size(0)
combined = torch.zeros(B, 32, device=x.device)

# 真正按专家做稀疏分发，而不是先算所有专家再相乘
for e_id, expert in enumerate(self.experts):
    selected_mask = (topk_idx == e_id).any(dim=1) # [B]
    if selected_mask.sum() == 0:
        continue

    h_sel = h[selected_mask] # [n_e, 64]
    out_sel = expert(h_sel) # [n_e, 32]
    w_sel = sparse_probs[selected_mask, e_id].unsqueeze(-1) # [n_e, 1]
    combined[selected_mask] += w_sel * out_sel

logits = self.head(combined)

lb_loss, lb_stats = switch_style_load_balance_loss(
    router_probs, topk_idx, self.num_experts, self.k
)

aux = {
    "router_probs": router_probs.detach(),
    "topk_idx": topk_idx.detach(),
    "topk_w": topk_w.detach(),
    "load_freq": lb_stats["f"],
    "mean_prob": lb_stats["P"],
    "lb_loss": lb_loss.detach(),
}
return logits, lb_loss, aux

```

src/train.py

```

def train_one_epoch(model, loader, optimizer, criterion, lambda_lb=0.01, grad_clip=1.0):
    model.train()
    total_loss, total_ce, total_lb = 0.0, 0.0, 0.0

    for xb, yb in loader:
        xb, yb = xb.cuda(), yb.cuda()

        out = model(xb)
        if isinstance(out, tuple): # MoE
            logits, lb_loss, aux = out
            ce_loss = criterion(logits, yb)

```

```

        loss = ce_loss + lambda_lb * lb_loss
    else:
        # Dense MLP
        logits = out
        ce_loss = criterion(logits, yb)
        lb_loss = torch.tensor(0.0, device=xb.device)
        loss = ce_loss

    optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
    optimizer.step()

    total_loss += loss.item() * xb.size(0)
    total_ce += ce_loss.item() * xb.size(0)
    total_lb += lb_loss.item() * xb.size(0)

    return {
        "loss": total_loss / len(loader.dataset),
        "ce": total_ce / len(loader.dataset),
        "lb": total_lb / len(loader.dataset),
    }

```

```

# src/train.py
@torch.no_grad()
def evaluate(model, loader, criterion, lambda_lb=0.01):
    model.eval()
    all_y, all_pred = [], []
    all_topk_idx, all_router = [], []

    total_loss = 0.0
    for xb, yb in loader:
        xb, yb = xb.cuda(), yb.cuda()

        out = model(xb)
        if isinstance(out, tuple):
            logits, lb_loss, aux = out
            loss = criterion(logits, yb) + lambda_lb * lb_loss
            all_topk_idx.append(aux["topk_idx"].cpu().numpy())
            all_router.append(aux["router_probs"].cpu().numpy())
        else:
            logits = out
            loss = criterion(logits, yb)

        pred = logits.argmax(dim=1)

        total_loss += loss.item() * xb.size(0)
        all_y.append(yb.cpu().numpy())
        all_pred.append(pred.cpu().numpy())

    y_true = np.concatenate(all_y)

```

```

y_pred = np.concatenate(all_pred)

result = {
    "loss": total_loss / len(loader.dataset),
    "acc": accuracy_score(y_true, y_pred),
    "report": classification_report(y_true, y_pred, output_dict=True, zero_division=0),
    "cm": confusion_matrix(y_true, y_pred, normalize="true"),
    "y_true": y_true,
    "y_pred": y_pred,
}

if len(all_topk_idx) > 0:
    result["topk_idx"] = np.concatenate(all_topk_idx, axis=0)
    result["router_probs"] = np.concatenate(all_router, axis=0)

return result

```

```

# src/viz.py
def build_expert_heatmap(y_true, topk_idx, num_classes=4, num_experts=4):
    # 行: 类别, 列: 专家
    heat = np.zeros((num_classes, num_experts), dtype=float)
    for c in range(num_classes):
        class_mask = (y_true == c)
        denom = max(class_mask.sum(), 1)
        for e in range(num_experts):
            routed = np.any(topk_idx[class_mask] == e, axis=1)
            heat[c, e] = routed.sum() / denom
    return heat

def plot_heatmap(mat, row_labels, col_labels, title, save_path):
    fig, ax = plt.subplots(figsize=(6, 4))
    im = ax.imshow(mat, aspect="auto")
    ax.set_xticks(range(len(col_labels)))
    ax.set_xticklabels(col_labels)
    ax.set_yticks(range(len(row_labels)))
    ax.set_yticklabels(row_labels)
    ax.set_title(title)
    for i in range(mat.shape[0]):
        for j in range(mat.shape[1]):
            ax.text(j, i, f"{mat[i,j]:.2f}", ha="center", va="center")
    plt.colorbar(im, ax=ax)
    plt.tight_layout()
    plt.savefig(save_path, dpi=200)
    plt.close()

```

对课程作业来说，还有一个非常实用的技巧：**MoE forward 一定要返回 diagnostics**。也就是上面 `aux` 里那些 `router_probs`、`topk_idx`、`load_freq`。没有这些统计，你后面的专家热图、门控分布、负载分析都会补不出来，实验报告就只剩“准确率高低”的空壳。 17

实验对比与四页报告撰写要点

实验对比计划建议非常清楚地固定为三组：**Dense MLP**、**MoE-K=1**、**MoE-K=2**。三者必须共享同样的预处理、同样的数据切分、同样的随机种子方案、同样的训练轮数上限、同样的早停规则。由于稀疏模型与稠密模型的参数总量概念并不完全可比，报告中最好同时记录**总参数量**与**单样本激活参数量**，但结论部分优先比较测试性能、训练稳定性与专家分工可解释性，而不要只看总参数数值。Hugging Face 的 MoE 中文说明明确提醒过，直接比较稀疏模型与稠密模型的参数数量并不恰当。 ²⁶

你可以直接在报告中放下面三张结果表模板。

总结果表模板

模型	专家数 E	K	总参数量	激活参数量/样本	Test Acc	Macro F1	Balanced Acc	CV(load)	训练时间/epoch
Dense MLP	-	-						-	
MoE	4	1							
MoE	4	2							

每类指标表模板

模型	类别	Precision	Recall	F1	Support
Dense MLP	C0				
Dense MLP	C1				
Dense MLP	C2				
Dense MLP	C3				
MoE-K=1	C0				
MoE-K=1	C1				
MoE-K=1	C2				
MoE-K=1	C3				
MoE-K=2	C0				
MoE-K=2	C1				
MoE-K=2	C2				
MoE-K=2	C3				

专家负载分布表模板

模型	Expert 0	Expert 1	Expert 2	Expert 3	max/mean	mean router prob
MoE-K=1						
MoE-K=2						

专家分工现象的解释，建议围绕“**标签分工**”与“**特征区域分工**”两条线写。第一种最直观：如果热图接近对角线，说明某些专家更稳定地对应某类击球。第二种更常见也更有研究味：专家未必严格按标签分工，而是按**速度区间、站位深浅、落点区域、是否靠近网前**这些子模式分工。比如一个专家主要接收“低球速、近网、短落点”的样本，它可能在语义上更接近“轻吊/丁克风格专家”；另一个专家主要接收“高球速、深落点、站位较后”的样本，则更像“平抽/强攻风格专家”。这类“按特征机制而不是按标签名字分工”的现象，与 MoE 文献中“专家会自然专门化”的观察是一致的。²⁵

关于“负载均衡是否必要”，报告里最好避免写成绝对句。更准确的讨论方式是：**必要，但不宜过强**。Shazeer 明确指出，门控网络会偏向少数专家，而且这种偏置会自我强化；这说明没有平衡项时，的确存在专家塌缩与 dead expert 风险。另一方面，如果 λ_{lb} 太大，门控会被强行推向均匀分配，可能压制本来有意义的专业化。因此，你可以在报告中把它写成一个张力：**平衡项在“防止塌缩”和“保留分工”之间找折中**。最实用的经验判据是：若某专家长期低于 5% 路由或某专家长期超过 60% 路由，通常说明平衡机制需要调整；若各专家都很平均但热图没有任何结构，则可能平衡过强。²⁷

MoE 相对稠密 MLP 的优缺点，建议在结论中概括成下表。

维度	稠密 MLP	稀疏 MoE
实现复杂度	低	高
训练稳定性	高	较低，依赖路由与平衡
解释性	中等，多靠特征重要性	较高，可分析专家分工
小数据鲁棒性	往往更稳	可能过拟合或塌缩
异质样本建模	一套参数统管全部	更适合形成子模式专家
适合作业演示	非常适合做基线	适合展示机制与分析深度

这张对照背后的共识是：表格数据上不存在“永远最优”的统一深度模型，强基线 MLP 很重要；而稀疏 MoE 的优势更多体现在输入异质性、条件计算和可解释分工上。²⁸

如果你需要把正式实验报告压到 **不超过 4 页**，一个比较稳妥的页面分配如下。

页面区域	建议内容
第 1 页	任务定义、数据说明、预处理、MLP 与 MoE 结构图（可放 mermaid 导出图）
第 2 页	训练配置表、损失函数公式、K=1/K=2 对比设定
第 3 页	总结果表 + 混淆矩阵
第 4 页	专家分工热图 + 现象解释 + 负载均衡讨论 + 优缺点总结

最后给出一份适合写进“推荐阅读”小节的文献表。下面这些引用本身即可作为可点击阅读链接。

推荐阅读	用途
Shazeer et al., Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer ²⁹	稀疏门控、Noisy Top-K、importance/load loss 原始来源
Lepikhin et al., GShard ³⁰	top-2 路由、容量与大规模专家并行
Fedus et al., Switch Transformers ³¹	top-1 路由与简化负载均衡损失
史宏志等, 《大模型时代的混合专家系统优化综述》 ³²	中文综述, 适合报告文献综述部分引用
Hugging Face 中文博客《混合专家模型详解》 ³³	适理解 top-1 / top-2、容量、稀疏与稠密取舍
Gorishniy et al., Revisiting Deep Learning Models for Tabular Data ³⁴	支撑“强 MLP 基线不可省略”这一点
匹克球技术与飞行动力学资料 ⁵	用于解释为何站位、速度、落点等特征是合理输入

综合以上方案, 最稳妥的实现路径是: 先做**强基线 MLP**, 再做**共享 stem + 4 专家 + Switch-style 负载均衡**的稀疏 MoE, 并在同一套预处理和训练配置下比较 **K=1 / K=2 / Dense MLP**; 报告中重点展示**混淆矩阵、专家热图和负载统计**, 再围绕“专家到底在按什么分工”“负载均衡项是否过强/过弱”两点做讨论。这种组织方式最符合课程作业的可执行性, 也最容易写出一份内容密度高、但仍能控制在 4 页内的实验报告。 ³⁵

¹ ⁴ ¹² ²⁸ ³⁴ ³⁵ <https://arxiv.org/abs/2106.11959>

<https://arxiv.org/abs/2106.11959>

² ⁸ https://scikit-learn.org/stable/common_pitfalls.html

https://scikit-learn.org/stable/common_pitfalls.html

³ ¹⁸ <https://jmlr.org/papers/volume23/21-0998/21-0998.pdf>

<https://jmlr.org/papers/volume23/21-0998/21-0998.pdf>

⁵ [https://ndltd.ncl.edu.tw/cgi-bin/gs32/gswweb.cgi/login?](https://ndltd.ncl.edu.tw/cgi-bin/gs32/gswweb.cgi/login?o=dnclcdr&s=id%3D%22109NTCP0419008%22.&searchmode=basic)

[o=dnclcdr&s=id%3D%22109NTCP0419008%22.&searchmode=basic](https://ndltd.ncl.edu.tw/cgi-bin/gs32/gswweb.cgi/login?o=dnclcdr&s=id%3D%22109NTCP0419008%22.&searchmode=basic)

<https://ndltd.ncl.edu.tw/cgi-bin/gs32/gswweb.cgi/login?o=dnclcdr&s=id%3D%22109NTCP0419008%22.&searchmode=basic>

⁶ <https://www.cs.toronto.edu/~fritz/absps/jjnh91.pdf>

<https://www.cs.toronto.edu/~fritz/absps/jjnh91.pdf>

⁷ ²⁶ ³³ <https://huggingface.co/blog/zh/moe>

<https://huggingface.co/blog/zh/moe>

⁹ <https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html>

<https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html>

¹⁰ ²⁰ <https://docs.pytorch.org/docs/2.12/generated/torch.nn.CrossEntropyLoss.html>

<https://docs.pytorch.org/docs/2.12/generated/torch.nn.CrossEntropyLoss.html>

¹¹ <https://www.sfu.ca/~tswartz/papers/emond.pdf>

<https://www.sfu.ca/~tswartz/papers/emond.pdf>

¹³ <https://docs.pytorch.org/docs/2.12/generated/torch.nn.BatchNorm1d.html>

<https://docs.pytorch.org/docs/2.12/generated/torch.nn.BatchNorm1d.html>

14 15 16 17 25 27 <https://www.cs.toronto.edu/~hinton/absps/Outrageously.pdf>
<https://www.cs.toronto.edu/~hinton/absps/Outrageously.pdf>

19 <https://arxiv.org/pdf/2006.16668>
<https://arxiv.org/pdf/2006.16668>

21 <https://github.com/yandex-research/rtdl-revisiting-models>
<https://github.com/yandex-research/rtdl-revisiting-models>

22 https://pytorch.org/ignite/generated/ignite.handlers.early_stopping.EarlyStopping.html
https://pytorch.org/ignite/generated/ignite.handlers.early_stopping.EarlyStopping.html

23 https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html

24 https://matplotlib.org/stable/gallery/images_contours_and_fields/image_annotated_heatmap.html
https://matplotlib.org/stable/gallery/images_contours_and_fields/image_annotated_heatmap.html

29 <https://arxiv.org/abs/1701.06538>
<https://arxiv.org/abs/1701.06538>

30 <https://arxiv.org/abs/2006.16668>
<https://arxiv.org/abs/2006.16668>

31 <https://arxiv.org/abs/2101.03961>
<https://arxiv.org/abs/2101.03961>

32 <https://crad.ict.ac.cn/cn/article/pdf/preview/10.7544/issn1000-1239.202440016.pdf>
<https://crad.ict.ac.cn/cn/article/pdf/preview/10.7544/issn1000-1239.202440016.pdf>